



Webfejlesztési keretrendszerek

10. gyakorlat

Dr. Jánki Zoltán Richárd

Hol tartunk most?

SZOFTVERFEJLESZTÉSI ÉLETCIKLUS ÉS FOLYAMAT



Legfontosabb okok

- Regresszió-védelem
 - Új feature ne tegye tönkre a meglévőeket
- Élő dokumentáció
 - Megmutatja, hogy a kódnak mit kell csinálnia
- Refaktor-bátorság
 - Hozzá merjünk nyúlni a kódhoz
- Bizalom a deploy előtt
 - “Ne hajnali 5-kor derüljön ki, hogy nem megy a login



Új kihívások a tesztelésnél

- AI gyorsan ír tesztet
 - nagy mennyiségű kód áttekintése
- AI nem feltétlenül érti a teljes rendszert
 - ezt kell elérni(!)
 - ügynökök
 - [ai-digest](#)
- AI magabiztosan hibázik
 - egyetlen automatizált módszer az AI-által írt kód tesztelésére



Tesztelési piramis

FRONTEND ALKALMAZÁS TESZTELÉSI PIRAMIS



GYORS VISSZAJELZÉS

Unit tesztek → azonnali visszajelzés a fejlesztés során



MEGBÍZHATÓSÁG

Integrációs tesztek → biztosítják az együttműködést

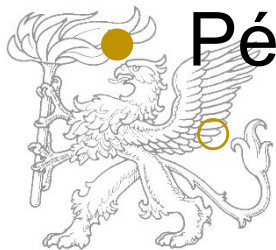


BIZALOM A FELHASZNÁLÓI ÉLMÉNYBEN

E2E tesztek → értékes felhasználói folyamatok védelme

Unit tesztek

- Mit tesztel?
 - egyetlen függvényt/metódust
 - egy komponens egy kis szeletét (izoláltan)
- Külső függőségek
 - mock-olva (HTTP, Firstore, Router, stb.)
- Sebesség
 - milliszekundumok alatt

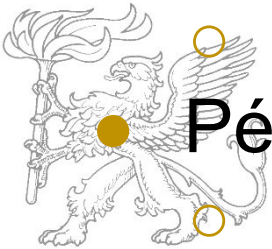


● Példa:

- ha egy átlagszámítás üres tömböt kap, akkor ne NaN-t adjon vissza, hanem 0-t

Integrációs tesztek

- Mit tesztel?
 - több egység együttműködését
 - komponens + service, service + service
- Külső függőségek
 - részben mock-olva
 - service-ek valódi példányban futnak
- Sebesség
 - másodpercen belül
- Példa:
 - a komponens betölti az adatokat a service-en keresztül és megjeleníti őket



End-to-End (E2E) tesztek

- Mit tesztel?
 - valódi felhasználói folyamatot valódi böngészőben (valódi backend-del)
- Külső függőségek
 - nincs mock-olva semmi
- Sebesség
 - másodpercek alatt

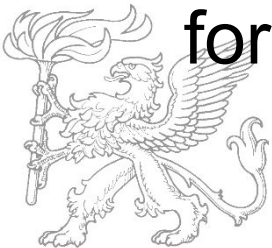


● Példa:

- a felhasználó bejelentkezik, összeállítja a heti étrend tervezetét, legenerálja a bevásárlólistát, majd kilép

“Ice Cream Cone”

- Fordított tesztelési piramis
 - rengeteg E2E teszt, kevés integrációs teszt, alig néhány unit teszt
- Gyakori AI hiba
 - meg kell mondani, hogy milyen tesztet szeretnék
- 95%-ban unit szinten kell tesztelni, nem fordítva



Teszt anatómia: AAA minta

AAA MINTA

A JÓ TESZTEK HÁROM LÉPÉSE



ARRANGE

készítsd elő az adatokat
és a függőségeket



ACT

hívd meg a
tesztelendő dolgot



ASSERT

ellenőrizd
az eredményt

PÉLDA (PSZEUDOKÓD)

```
1 // Arrange
2 const ratings = [4, 5, 3];
3
4 // Act
5 const avg = service.average(ratings);
6
7 // Assert
8 expect(avg).toBe(4);
```



MAGYARÁZAT

Minden jó teszt ezt a három lépést követi, függetlenül attól, hogy **Jasmine**, **Vitest** vagy **Jest**.

Ha egy tesztben össze van keverve a három, az nehezen olvasható.

Az AI által generált tesztekben gyakran látható az is, hogy több tesztet egybegyűr — ezt szét kell szedni **egy teszt = egy állítás** logika szerint.



CÉL: Jól olvasható, karbantartható és megbízható tesztek.

Tesztelési stack-ek

JAVASCRIPT TESZTELÉSI ESZKÖZÖK ÖSSZEHAISONLÍTÁSA

TÉMA	 ANGULAR	 REACT	 VUE
 TEST RUNNER / FRAMEWORK	Jasmine + Karma (újabbán Jest / Vitest is)	Jest (vagy Vitest)	Vitest (vagy Jest)
 KOMPONENS- TESZTELÉS	TestBed	React Testing Library (RTL)	Vue Test Utils
 DOM ASSERTION KÖNYVTÁR	beépített (Jasmine matchers)	<code>@testing-library/ jest-dom</code>	<code>@testing-library/ jest-dom</code>
 MOCKING	Jasmine spies / <code>createSpyObj</code>	<code>jest.fn()</code> / <code>vi.fn()</code>	<code>vi.fn()</code> / <code>jest.fn()</code>
 E2E	Cypress / Playwright	Cypress / Playwright	Cypress / Playwright
 COVERAGE	<code>ng test --code-coverage</code>	<code>jest --coverage</code>	<code>vitest --coverage</code>
 CLI PARANCs (ALAP)	<code>ng test</code>	<code>npm test</code>	<code>npm run test:unit</code>

Mocking és Spying

FÜGGŐSÉGEK KONTROLLÁLT TESZTELÉSE



MOCK

kicseréljük a valódi függőséget egy hamisra



SPY

figyeljük, hogy egy metódust meghívtak-e, milyen paraméterrel



MAGYARÁZAT

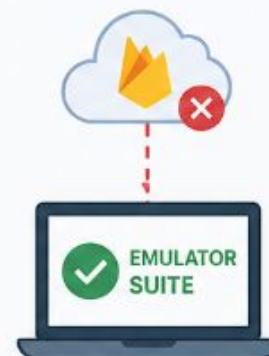
Ez az a pont, ahol megáll az ember, mert „de hát a service-em Firestore-tól függ”.

Mock-olással kicseréljük.

Soha ne hívj valódi Firestore-t unit tesztből, mert:

- 1 lassú
- 2 flakey lehet (hálózati hiba)
- 3 szennyezi az adatbázist

A Firebase Emulator Suite kifejezetten erre is jó — de az inkább integrációs teszt szintje.



RECEPTKATALÓGUS PÉLDA: `RecipeService.delete()`

1. A SERVICE (ÉLES KÓD)

```
import { deleteDoc, doc } from 'firebase/firestore';

export class RecipeService {
  constructor(private db: Firestore) {}

  async delete(id: string) {
    const ref = doc(this.db, 'recipes', id);
    await deleteDoc(ref);
  }
}
```

2. UNIT TESZT: MOCK + SPY

```
it('meghívja a deleteDoc-ot a helyes útvonallal', async () => {
  // Arrange - mock + spy
  const deleteDocSpy = vi.fn().mockResolvedValue(undefined);
  const docSpy = vi.fn((db, collection, id) => `recipes/${id}`);

  const mockDb = {};
  const service = new RecipeService(mockDb as any);

  vi.mock('firebase/firestore', () => ({
    deleteDoc: deleteDocSpy,
    doc: docSpy,
  }));
  // Act
  await service.delete('abc123');
  // Assert - spy: ellenőrizzük a meghívást és a paramétert
  expect(docSpy).toHaveBeenCalledOnceWith(mockDb, 'recipes', 'abc123');
  expect(deleteDocSpy).toHaveBeenCalledOnceWith(`recipes/abc123`);
});
```

3. EREDMÉNY

- ✓ A Firebase Firestore nem kerül meghívásra.
- ✓ Gyors, stabil (nem függ a hálózattól).
- ✓ Csak azt teszteljük, amit kell: a service helyesen hívja-e meg a függőséget.



LÉNYEG: Mock = helyettesítünk. Spy = megfigyelünk.
Ezzel izoláljuk az unit tesztet és megbízhatóvá tesszük az eredményeket.

Guard-ok tesztelése

UNIT TESZT – BIZTONSÁG KRITIKUS LOGIKA ELLENŐRZÉSE



ARRANGE

AuthService mock,
ami `isAdmin` jelet ad vissza



ACT

hívjuk a guard-ot



ASSERT

ha nem admin → `false` és
`Router.navigate('/login')`
meghívva



MAGYARÁZAT

A guard-ok kritikus biztonsági pontok.

Ha a guard tesztet írunk, az pontosan az a logika, amit az értékelő rendszer is keres (security szempontnál is, tesztelés szempontnál is).

Egyetlen guard teszt-fájl 3-4 unit tesztet ér:

- 1 bejelentkezett admin → enged
- 2 bejelentkezett user → tilt
- 3 anonim → redirect



A receptkatalógusban van legalább kettő ilyen guard:
az `AuthGuard` (csak bejelentkezett user értékelhet) és
az `AdminGuard` (csak admin tölthet fel receptet).

PÉLDA TESZT KÓD (Vitest/Jest + Angular)

ARRANGE

```
// Arrange
const authServiceMock = {
  isAdmin: vi.fn(() => false)
};
const routerMock = {
  navigate: vi.fn()
};
const guard = new AdminGuard(
  authServiceMock as any,
  routerMock as any
);
```

ACT

```
// Act
const result = guard.canActivate(
  {} as ActivatedRouteSnapshot,
  {} as RouterStateSnapshot
);
```

ASSERT (nem admin eset)

```
// Assert
expect(result).toBe(false);
expect(routerMock.navigate)
  .toHaveBeenCalledWith(['/login']);
```



CÉL: A guard csak akkor engedjen tovább, ha admin.
Ellenkező esetben tiltsa és irányítsa a /login oldalra.

TIPIKUS UNIT TESZTEK – ADMIN GUARD



1. BEJELENTKEZETT ADMIN → ENGED

- ✓ `isAdmin = true` → `true`-t ad vissza
- ✓ `router.navigate` nincs meghívva



2. BEJELENTKEZETT USER → TILT

- ✓ `isAdmin = false` → `false`-t ad vissza
- ✓ `router.navigate('/login')` meghívva



3. ANONIM → REDIRECT

- ✓ nincs user → `false`-t ad vissza
- ✓ `router.navigate('/login')` meghívva



Code Coverage

- A kód hány százaléka fut le a tesztek alatt?
- “Lustasági” mutató
- Nem mond semmit a tesztek minőségéről
- Reális cél: 70-80% (nem 100%(!))



CI/CD

A SZÁMÍTÓGÉP TARTJA BE A SZABÁLYOKAT, NEM TE.



CI (CONTINUOUS INTEGRATION)

Minden push-nál automatikusan lefutnak a tesztek, lint, build.



1. CODE PUSH

Fejlesztő
commit-ol



2. CI TRIGGER

Automatikusan
elindul



3. PIPELINE

Teszt, lint, build
lépések lefutnak



4. CI EREDMÉNY

Zöld = sikeres
piros = hiba



CD (CONTINUOUS DELIVERY / DEPLOYMENT)

Ha a CI zöld, automatikusan élesedik
(vagy gombnyomásra).



CI ZÖLD

A pipeline
sikeres



DEPLOY

Automatikus
vagy kézi jóváhagyás



ÉLES

Új verzió
élesben



MIÉRT?

„Nálam működött” kifogás
megszüntetése.



Mindenki ugyanazt
a folyamatot futtatja



Ha eltörik valami,
mindenki azonnal látja



Nem kell várni kézi
ellenőrzésre



Gyorsabb, megbízhatóbb
és kiszámíthatóbb



ESZKÖZÖK

- GitHub Actions
- GitLab CI
- CircleCI
- ...



PUSH / MERGE

Pull request



CI TRIGGER

Pipeline indul



LINT

Kódelemzés



TESTS

Unit / E2E



BUILD

Alkalmazás építése



CI ZÖLD

Minden ok



DEPLOY

Automatikus
vagy kézi



ÉLES

Felhasználóknál



A LÉNYEG

Bárki commit-ol, a pipeline automatikusan lefut: tesztek, lint, build.

Ha valami hibás, azonnal fény derül rá – nem kell várni senki kézi ellenőrzésére.

Build, Deploy, Release

Build ≠ Deploy ≠ Release



BUILD

A kódból futtatható artifact

A fejlesztői kód fordítása, csomagolása, amin egy futtatható változat jön létre. Ez még nem kerül éles környezetbe.

PÉLDA



dist/ mappa
vagy .jar, .zip, .apk, Docker image stb.



FONTOS

A build az, amit a CI/CD pipeline előállít. Futtatható, de még sehol nem fut.



DEPLOY

Az artifact felkerül a production szerverre

A build artifact telepítése az éles (vagy előéles) környezetre. A rendszer már elérhető, de a felhasználók még nem feltétlenül látják.

PÉLDA



Az új verzió fut a szerveren, de lehet, hogy csak az adminok vagy 5% látja.



FONTOS

A deploy technikai művelet. Gyorsan, gyakran, akár naponta többször.



RELEASE

A felhasználók valóban használják az új verziót

A változás elérhetővé válik a felhasználók számára. Ekkor történik a valódi „élesítés”.

PÉLDA



A felhasználók az új funkciót látják és használják.



FONTOS

A release üzleti döntés. Megfontolt, kontrollált, visszavonható.

A FOLYAMAT IDŐBEN



KÓD

Fejlesztői kód



BUILD

CI pipeline létrehozza az artifactet (pl. dist/)



DEPLOY

Az artifact felkerül a production szerverre



RELEASE

A felhasználók megkapják és használják az új verziót



FELHASZNÁLÓK

Valódi értékhez jutnak



LÉNYEG

Lehet deploy-olni úgy, hogy a felhasználó nem lát semmit (pl. canary, feature flag mögé rejtett funkció).

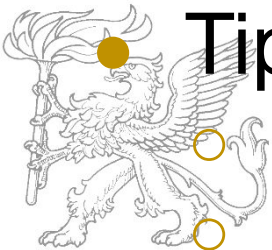
Deploy gyorsan.
Release megfontoltan.

Production-ready audit

- Lighthouse (Chrome DevTools)
 - Teljesítmény (LCP, FID, CLS)
 - Akadálymentesség
 - Legjobb gyakorlatok
 - SEO
- Cél: 90+ minden kategóriában

Tipikus problémák

- Lazy Loading hiánya → nagy első bundle
- Képek nincsenek optimalizálva → LCP rossz
- Heading hierarchia kihagyva → Rossz a11y



Futtatás előtti ellenőrzési lista

- console.log()-ok kitisztítva
- TODO-k átnézve
- .gitignore rendben (nincs .env, nincs node_modules)
- environment fájlok produkcióra állítva
- tesztek “zöldek”
- Lighthouse 90+
- Readme.md naprakész



Gyakorlat

- Tesztelés
 - specifikáció mentén teszt tervezés
 - tesztek implementálása
- Production-ready audit és kitelepítés előtti ellenőrzés
- Alkalmazás kitelepítése (deploy)

